



# Vinci Automation

## Phased Development Plan

One build phase per Product Requirements Document, sequenced from the platform foundation through to the day-to-day operations layer. Each phase sets out its pre-requisites, goal, step-by-step build process, final deliverable, and how to evaluate that the phase is done.

|                           |                                                                                                                      |
|---------------------------|----------------------------------------------------------------------------------------------------------------------|
| <b>Primary source</b>     | The six Vinci Automation PRDs (Overall, Chatbot & Data Input, Lesson Calendar, Lesson Dashboard, Financials, Urgent) |
| <b>Supporting context</b> | Vinci AI Automation Business Rules v1.2 — referenced for technical grounding only, never overriding PRD scope        |
| <b>Sequencing logic</b>   | Foundation → data entry → visualization → assignment workflow → reporting → cross-cutting exception handling         |

# Phase Map

Six PRDs, six build phases, in development order.

|   |                                         |                      |
|---|-----------------------------------------|----------------------|
| 1 | Platform Foundation, Login & Navigation | Overall              |
| 2 | Data Input & Chatbot Assistant          | Chatbot & Data Input |
| 3 | Lesson Calendar                         | Lesson Calendar      |
| 4 | Lesson Dashboard                        | Lesson Dashboard     |
| 5 | Financials Hub                          | Financials           |
| 6 | Urgent Situations Page                  | Urgent               |

**Why this order:** the Overall PRD defines the login gate and page shell every other feature lives inside, so it ships first. Data Input follows immediately because it is the PRD's only source of new lessons, courses, teachers and schools — every other page depends on data that enters through it. The Lesson Calendar and Lesson Dashboard then give the admin ways to view and act on that data. Financials can only report on lessons that have actually run, so it comes after the operational pages. The Urgent page is last because it surfaces cancellations, conflicts, and unassigned lessons that are detected across all of the systems built in Phases 1–5.

Note on scope: per Business Rules v1.2 (Section 1, Feature Scope by Version), the LLM chatbot itself and the automated reschedule flow are flagged for a later version (V2), while data input, scheduling, the calendar, and month-end finance snapshots are V1. This is flagged inline in Phase 2 as supporting context; the phase below still follows the Chatbot & Data Input PRD's full scope as the primary source.

## PHASE 1 OF 6

# Platform Foundation, Login & Navigation

Source PRD: Vinci Automation (Overall) PRD

## PRE-REQUISITES

- Hosting/framework decision made, since every other phase builds its pages inside this shell.
- Database scaffolded with at least empty tables for Schools, Courses, Teachers and Lessons, so pages have something to point at.
- Auth provider configured for Google OAuth and email + password sign-in.
- Final list of sidebar pages and labels agreed: Calendar, Lesson Dashboard, Urgent, Financials, plus the Chat/Data Input panel.

## GOAL

*Stand up the entry point and persistent page shell for Vinci Automation: a login gate that only validates Vinci-domain emails as admin, a left-hand navigation sidebar, and a collapsible chatbot/data-input panel on the right — the structure every later phase will be built inside of.*

## STEP-BY-STEP PROCESS

1. Build the login screen and validate the signed-in email's domain; only addresses on the Vinci domain are granted admin status.
2. Build the "Invalid login" fallback page shown to any non-Vinci email that attempts to sign in.
3. Persist the admin session so a successful login lands the user inside the main app rather than back at login.
4. Build the left sidebar navigation: clear labels and icons for each page, with a visible indicator for whichever page is currently open.
5. Build the main two-zone layout: the main content area on the left/center (defaulting to the lesson calendar) and the chatbot/data-input panel docked to the right.
6. Add the toggle inside the right panel to switch between "Chat" and "Data Input" subviews, plus a collapse/expand control so the admin can widen the main view.
7. Wire each sidebar link to its page route (pages themselves are filled in during later phases) so the navigation skeleton is fully clickable end to end.
8. Smoke-test both paths: a Vinci email reaches the shell with the calendar placeholder and chatbot panel visible; a non-Vinci email is stopped at the invalid-login page.

## FINAL DELIVERABLE

A deployed, navigable application shell: working login gate (Vinci-domain admin access only), a left sidebar that links to every page, and a collapsible right-hand chatbot/data-input panel — ready for Phases 2–6 to build their pages into.

## EVALUATION

- ✓ A Vinci-domain email logs in successfully and lands on the shell; any other email is shown the invalid-login page and gets no further access.
- ✓ The sidebar correctly highlights whichever page is currently open as the admin navigates between them.
- ✓ The chatbot/data-input panel can be collapsed and re-expanded without breaking or resizing the main content area incorrectly.

## PHASE 2 OF 6

## Data Input & Chatbot Assistant

Source PRD: Vinci Automation (Chatbot & Data Input) PRD

### PRE-REQUISITES

- Phase 1 shell complete, with the right-hand panel's Chat / Data Input toggle already in place.
- Database tables for Lessons, Courses, Teachers and Schools exist, with an agreed fixed ID-generation template for each.
- WhatsApp (WATI) integration credentials available, since submitting a new lesson must trigger an offer-blast message and the system must be able to receive accept replies.

### GOAL

*Give the admin the one and only place new data enters the system — lessons, courses, teachers, schools — and a chatbot that can summarize existing data or make simple modifications to it on request.*

### STEP-BY-STEP PROCESS

1. Build the four-tab Data Input subview (Lesson, Course, Teacher, School), each with the exact text-input fields the PRD specifies and every field marked mandatory.
2. On submit, validate that all fields are filled, generate the record's ID from the fixed template, and write the new record directly into the database.
3. For the Lesson tab specifically: on submit, set the new lesson's status to Unassigned, trigger the WhatsApp offer blast to teachers matching availability, and confirm the lesson appears immediately on the calendar (Phase 3) colored yellow or red depending on how soon it falls.
4. Build the chatbot chat box: admin types a prompt, the bot reads from the database, and returns a clean, readable summary (e.g. "all lessons this week and their statuses").
5. Scope the chatbot's write ability to modification only — e.g. changing a lesson's date — and explicitly block it from creating new records or deleting existing ones.
6. After a chatbot-driven change, write it to the database directly and have the bot reply in-chat confirming exactly what was changed.
7. Add the two failure-handling behaviors: a popup telling the admin to refresh/reopen before retrying on a failed data input, and a basic error message after repeated chatbot failures.

### FINAL DELIVERABLE

A working four-tab data-input form that is the sole way new lessons, courses, teachers and schools enter the system, plus a chatbot that can summarize current data and modify (but never create or delete) existing records, with every change reflected live across the rest of the app.

### EVALUATION

- ✓ Submitting a new lesson appears correctly colored on the calendar and triggers a WhatsApp offer blast to eligible teachers.
- ✓ Submitting a form with any field left blank is blocked, since every input field is mandatory.
- ✓ Asking the chatbot for a weekly lesson summary returns an accurate, readable result.
- ✓ Asking the chatbot to reschedule a lesson updates the database and the calendar; asking it to delete or create a record is refused.

*Context from Business Rules v1.2: the LLM chatbot itself is flagged as a V2 feature (data input, scheduling and the offer blast are V1). This phase still delivers the Chatbot & Data Input PRD in full; if timeline pressure is high, the data-input tabs can ship first and the chatbot can launch with a lighter, hardcoded-summary fallback without blocking the rest of the phase.*

## PHASE 3 OF 6

# Lesson Calendar

Source PRD: Vinci Automation (Lesson Calendar) PRD

## PRE-REQUISITES

- Lessons exist in the database (entered via Phase 2) with date, time and tutor-assignment status.
- Color-status logic available: red, yellow, or green depending on assignment and how close the lesson date is.
- A reusable slideout/detail-panel UI component, since several later pages (Dashboard, Urgent) reuse this pattern.

## GOAL

Give the admin an easy-to-read, color-coded calendar of every scheduled lesson across Day, Week and Month views, with click-through to full lesson detail and quick WhatsApp actions.

## STEP-BY-STEP PROCESS

1. Build the Day / Week / Month view toggle and a "Today" jump button.
2. Render each lesson as a colored bar showing time and course name, using the rule set: red = unassigned and within a week, yellow = unassigned and more than a week out, green = tutor assigned.
3. Implement the three layout variants: Day as a single column, Week as seven day-columns, Month as 28–31 date boxes.
4. Implement click-to-expand: clicking a lesson bar slides a detail panel in from the right showing School, Date & Time, Assigned tutor(s), Number of tutors, Lesson Income, and Lesson Material Link.
5. Make Lesson Income and Lesson Material Link editable within the slideout; keep every other lesson field read-only to avoid accidental changes.
6. Add the two quick-action buttons inside the slideout: "Send WhatsApp New Lesson Assignment" (greyed out once enough tutors have accepted) and "Send Material & Reminder to Chosen Tutors."
7. Confirm calendar colors update live the moment a lesson's tutor-assignment status changes elsewhere in the app (from the Lesson Dashboard or Urgent page).

## FINAL DELIVERABLE

A mostly read-only calendar with working Day/Week/Month views, accurate color-coding, and a detail slideout with the two editable fields and two WhatsApp quick-action buttons.

## EVALUATION

- ✓ An unassigned lesson within a week shows red; the same lesson more than a week out shows yellow; once a tutor is assigned it turns green.
- ✓ Clicking any lesson bar opens the slideout with all six specified fields correctly populated.

- ✓ “Send WhatsApp New Lesson Assignment” is greyed out once a lesson already has enough accepted tutors.



## PHASE 4 OF 6

# Lesson Dashboard

Source PRD: Vinci Automation (Lesson Dashboard) PRD

## PRE-REQUISITES

- Lessons and their WhatsApp acceptance data are flowing into the database (Phases 2–3 already live).
- A tutor-scoring mechanism is available, so accepted tutors can be ranked highest score to lowest.

## GOAL

*Give the admin a focused, action-first table of only the lessons that still need a tutor, sorted by urgency, with one-click manual assignment.*

## STEP-BY-STEP PROCESS

1. Build the lesson rows table with Lesson ID, Course name, Date & Time, and an “Assign Tutor” button on the far right.
2. Implement the sort order: lessons with zero accepted tutors first, then lessons with accepted-but-unselected tutors, with each group sorted closest date first.
3. On “Assign Tutor” click, open a slideout listing every teacher who accepted that lesson via WhatsApp, sorted from highest score to lowest.
4. Implement the assignment action: selecting a tutor writes directly to the database, and the lesson drops off this page automatically once enough tutors are selected.
5. Add the warning pop-up shown when re-assigning a different tutor to an already-assigned lesson, and the error message shown when trying to assign the same tutor twice.
6. Verify a completed assignment here is reflected instantly on the Calendar (turns green) and clears the lesson from the Urgent page if it was flagged there.

## FINAL DELIVERABLE

A working unassigned-lessons table, sorted by urgency, with a score-sorted tutor-assignment slideout and the two required validation pop-ups.

## EVALUATION

- ✓ Lessons with zero accepted tutors always appear above lessons with accepted-but-unselected tutors, and each group is date-sorted within itself.
- ✓ Assigning the required number of tutors removes the lesson from the dashboard automatically.
- ✓ Re-assigning a different tutor surfaces a warning; assigning the same tutor twice is blocked with an error.

## PHASE 5 OF 6

## Financials Hub

Source PRD: Vinci Automation (Financials) PRD

### PRE-REQUISITES

- At least one full month of completed lessons exists (with income, hours and tutor rates recorded), so the figures aren't empty on first load.
- A monthly snapshot calculation (tutor salary + course profit/loss) is available to generate the previous month's figures.

### GOAL

*Give the user a clean, read-only hub to check tutor salary expenses and course profitability, defaulting to the most recent month with the ability to drill into any past month or an overall view.*

### STEP-BY-STEP PROCESS

1. Build the two-section page layout: Tutor Salary section first, Course Financials section below it, with a clear visual separation between them.
2. Build Tutor Salary rows: tutor name and contact first, then total salary, each row separated by a divider line, ordered most-important data first left to right.
3. Build Course Financials rows: course name first, then profit margin in both percentage and HKD, then the fuller breakdown of total income and total expenses.
4. Default the page to the most recently completed month's data on first load.
5. Add a dropdown to select any earlier month, and a separate button for the all-months "overall" view.
6. Lock the entire page read-only: confirm there are no input fields or edit controls anywhere on this page.
7. Confirm the monthly figures regenerate automatically on the 1st of each month for the prior month, while every older month stays stored and selectable.

### FINAL DELIVERABLE

A two-section, read-only Financials hub (Tutor Salary + Course Financials) with a month selector and an overall view, defaulting to the latest completed month.

### EVALUATION

- ✓ On open, the page shows last month's figures by default with Tutor Salary shown above Course Financials.
- ✓ Selecting an earlier month from the dropdown swaps in the correct historical figures; the overall button aggregates across every month.
- ✓ No field anywhere on this page can be edited.

## PHASE 6 OF 6

## Urgent Situations Page

Source PRD: Vinci Automation (Urgent) PRD

### PRE-REQUISITES

- The Lesson Dashboard (Phase 4) and Lesson Calendar (Phase 3) are complete, since this page reuses their assignment and detail components.
- WhatsApp reply parsing for cancellations and reschedule requests is live, so urgent items can be detected automatically rather than entered by hand.
- Backend checks for scheduling conflicts and data anomalies (double-booked tutors, missing data, no tutor within a week) are running on a schedule.

### GOAL

*Give the admin one place where every situation needing immediate attention — cancellations, reschedule conflicts, soon-unassigned lessons, double-booked tutors, and data anomalies — surfaces automatically with a clear count and a one-click fix.*

### STEP-BY-STEP PROCESS

1. Build the red notification bubble on the sidebar's Urgent icon, showing a live count of open issues.
2. Build the Urgent page itself: each issue gets its own section with a title naming the problem and a short brief on what happened and how to fix it.
3. Wire up automatic detection for each of the five categories: tutor cancellation, reschedule conflicts, lessons with no tutor within a week, conflicting tutor schedules, and missing or conflicting data.
4. For a tutor cancellation or reschedule conflict, surface the lesson with a manual re-assign action that opens the same tutor-pool picker used on the Lesson Dashboard.
5. Below the live issues, list lessons within the coming week that still have no tutor, so the admin can batch-assign without leaving the page.
6. Ensure every fix made here writes directly to the database, immediately clears that issue from the page, and decrements the notification count.
7. Schedule the recurring data-audit job that scans for missing or conflicting data and feeds any findings into this same page.

### FINAL DELIVERABLE

A live Urgent page with an accurate notification badge, one clearly explained section per active issue, an in-place fix for each (re-assign tutor, resolve conflict), and a batch-assignment list for soon-unassigned lessons.

### EVALUATION

- ✓ A simulated WhatsApp cancellation reply produces a notification-count increase and a matching section on the Urgent page within the expected detection window.
- ✓ Re-assigning a tutor from this page resolves the issue, removes its section, and decrements the badge count.
- ✓ A lesson within a week with no tutor assigned appears in the batch-assignment list at the bottom of the page.